

# Cybersecurity and AI Case Studies



**Module Name: Cybersecurity and AI Case Studies**

**Module Code: MOD006570, Element 010**

**Module Leader: Raj Shukla**

**Name: Reema Firdous**

**Std ID: 2453991**

## Contents

|                                                          |    |
|----------------------------------------------------------|----|
| <b>Part A</b> .....                                      | 3  |
| <b>1. Model Selection and Rationale</b> .....            | 3  |
| <b>2. Dataset Description</b> .....                      | 3  |
| <b>3. Data Preprocessing</b> .....                       | 4  |
| <b>4. Random Forest Model</b> .....                      | 5  |
| <b>5. Random Forest Results</b> .....                    | 5  |
| <b>6. Decision Tree Model</b> .....                      | 6  |
| <b>7. Decision Tree Results</b> .....                    | 6  |
| <b>8. Comparative Analysis and Model Selection</b> ..... | 7  |
| <b>Part B:</b> .....                                     | 8  |
| <b>1. Data Poisoning Strategy</b> .....                  | 8  |
| <b>2. Random Label Flipping</b> .....                    | 8  |
| <b>3. Targeted Label Flipping</b> .....                  | 10 |
| <b>4. Experimental Results</b> .....                     | 13 |
| <b>5. Critical Evaluation</b> .....                      | 14 |
| <b>6. Countermeasures</b> .....                          | 15 |
| <b>Part C</b> .....                                      | 16 |
| <b>1. Introduction</b> .....                             | 16 |
| <b>2. Professional Architecture Diagram</b> .....        | 16 |
| <b>3. Agent Description</b> .....                        | 17 |
| <b>4. Coordination Between Agents</b> .....              | 17 |
| <b>5. Security Mechanisms</b> .....                      | 18 |
| <b>6. Threat Mitigation</b> .....                        | 18 |
| <b>7. Relationship with Parts A and B</b> .....          | 18 |
| <b>8. Conclusion</b> .....                               | 18 |

**Part A:**

**1. Model Selection and Rationale**

The UNSW-NB15 dataset was used to choose two supervised machine learning algorithms for intrusion detection: **Random Forest and Decision Tree**. The goal was to find the model that achieved the best balance of attack detection performance and classification accuracy.

Random Forest was chosen as the primary model because it mixes numerous decision trees to improve prediction accuracy while reducing overfitting. It works effectively on high-dimensional datasets with both numerical and categorical variables, making it ideal for network intrusion detection. A Decision Tree model was also used as a baseline classifier because of its simplicity, quick training time, and interpretability. Comparing both models provides a clear justification for selecting the most suitable model for subsequent experiments.

Accuracy, Precision, Recall, F1-score, and the Confusion Matrix were all used to evaluate model performance. Because intrusion detection seeks to reduce undetected attacks, recall and false negatives were deemed the most essential evaluation criteria.

**2. Dataset Description**

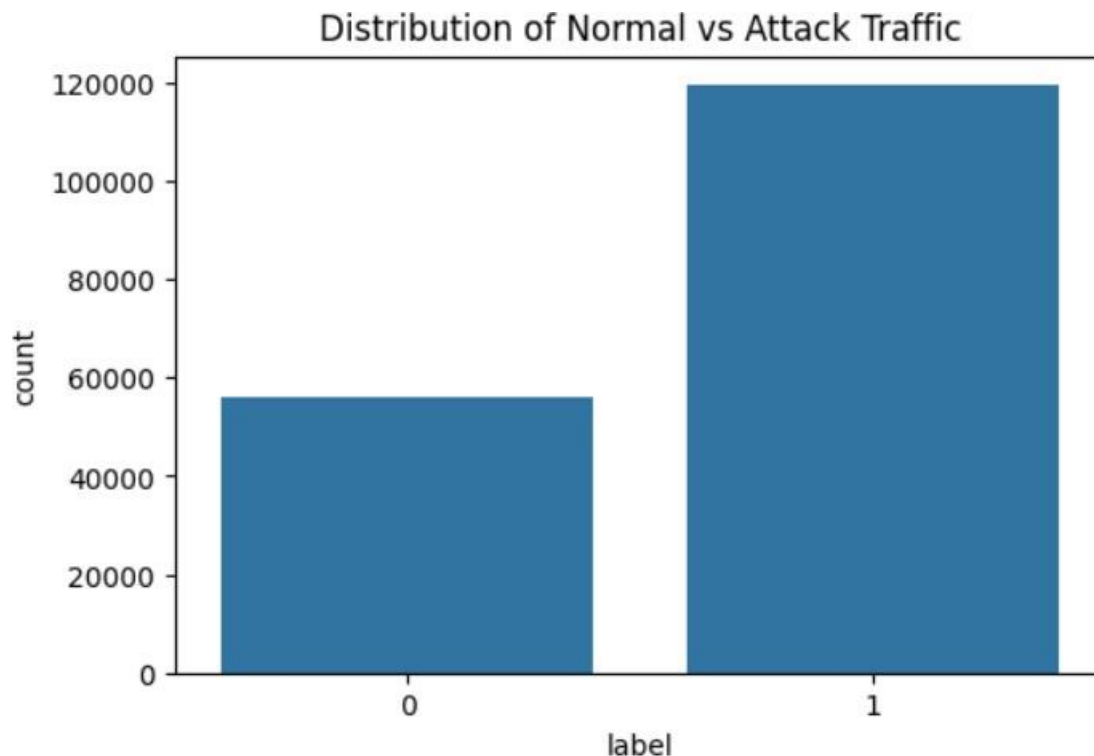
The experiments were conducted using the **UNSW-NB15** dataset, which contains modern network traffic representing both normal activities and malicious attacks. The dataset includes separate training and testing files, allowing the models to be trained and evaluated on independent data.

The original dataset contains **45 attributes**, including numerical and categorical features describing network traffic characteristics. The **label** column was used as the target variable, where **0 represents normal traffic** and **1 represents malicious traffic**. The **attack\_cat** column was excluded because the objective of this project is binary classification.

| Property            | Value                    |
|---------------------|--------------------------|
| Dataset             | UNSW-NB15                |
| Training Samples    | 175,341                  |
| Testing Samples     | 82,332                   |
| Original Features   | 45                       |
| Features Used       | 42                       |
| Target Variable     | Label                    |
| Classification Type | Binary (Normal / Attack) |

The training dataset includes **119,341** attack samples and **56,000** normal samples, demonstrating a moderate class imbalance favouring malicious traffic. There were no missing values discovered during data exploration, hence no imputation was required.

Figure 1 - Class Distribution of the Training Dataset.



### 3. Data Preprocessing

Several preprocessing steps were performed before training the models. The **id** column was removed because it does not contribute to prediction, while the **attack\_cat** column was removed since only binary classification was required.

The categorical features **proto**, **service**, and **state** were converted into numerical values using **Label Encoding**. During implementation, unseen categorical values were found in the testing dataset. This problem was handled by using a consistent encoding approach for both datasets.

| Dataset      | Samples | Features |
|--------------|---------|----------|
| Training Set | 175,341 | 42       |
| Testing Set  | 82,332  | 42       |

### 4. Random Forest Model

The pre-processed training dataset was used to train a Random Forest classifier, which was then assessed on the testing dataset. The model employs a collection of decision trees to improve prediction accuracy and reduce overfitting. Random Forest was chosen because it is reliable when dealing with large datasets and performs well in classification issues requiring high-dimensional data.

Accuracy, Precision, Recall, F1-score, and the Confusion Matrix were all used to evaluate model performance.

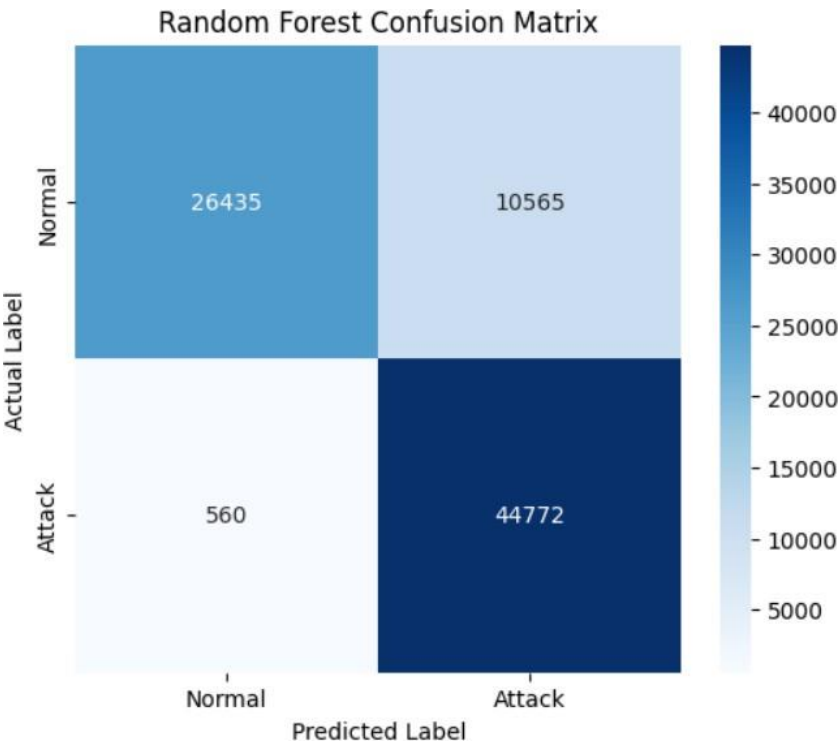
### 5. Random Forest Results

The Random Forest model had an overall accuracy of 86.49% on the testing dataset. A classification report is supplied.

| Class  | Precision | Recall | F1-score |
|--------|-----------|--------|----------|
| Normal | 0.98      | 0.71   | 0.83     |
| Attack | 0.81      | 0.99   | 0.89     |

**Overall Accuracy: 86.49%**

Figure 2 -Random Forest Confusion Matrix



The confusion matrix indicates that the model accurately categorised **44,772** attack samples while misclassifying **560** as regular traffic. This resulted in an attack recall of **99%**, suggesting that the model correctly detected almost all harmful activity.

However, **10,565** normal samples were misclassified as attacks, which increased the amount of false positives. In most intrusion detection systems, this trade-off is acceptable because identifying hostile traffic is more essential than eliminating false alerts.

## 6. Decision Tree Model

A Decision Tree classifier was developed using the same training and testing datasets to compare its performance with the Random Forest model. Decision Trees are easy to interpret and require less computational complexity, making them a suitable baseline classifier for this study.

The same evaluation metrics were used to ensure a fair comparison between both models.

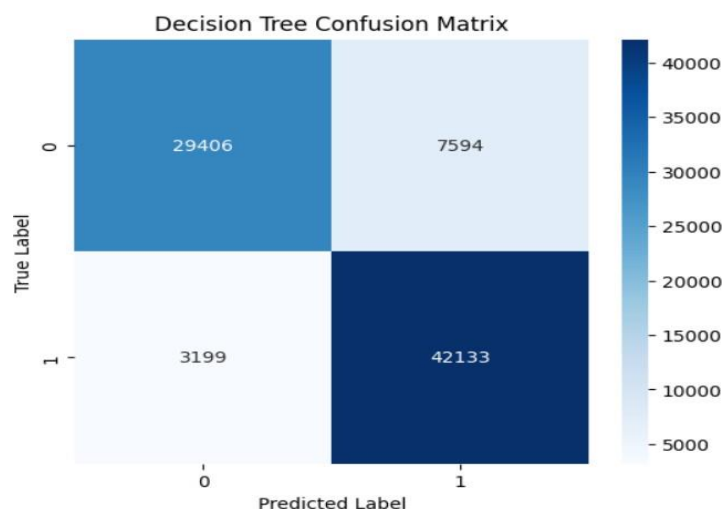
## 7. Decision Tree Results

The Decision Tree classifier has an overall accuracy of **86.89%**, slightly higher than the Random Forest model.

| Class  | Precision | Recall | F1-score |
|--------|-----------|--------|----------|
| Normal | 0.90      | 0.79   | 0.84     |
| Attack | 0.85      | 0.93   | 0.89     |

**Overall Accuracy: 86.89%**

Figure 3-Decision Tree Confusion Matrix



The Decision Tree accurately identified **42,133** assault samples, however **3,199** were wrongly classed as regular traffic. Although the model had a slightly greater overall accuracy, the attack recall dropped to **93%**, resulting in much more false negatives than the **Random Forest model**.

False negatives in intrusion detection signify unreported attacks and have a higher security impact than false positives. As a result, recall is prioritised over overall accuracy in this application.

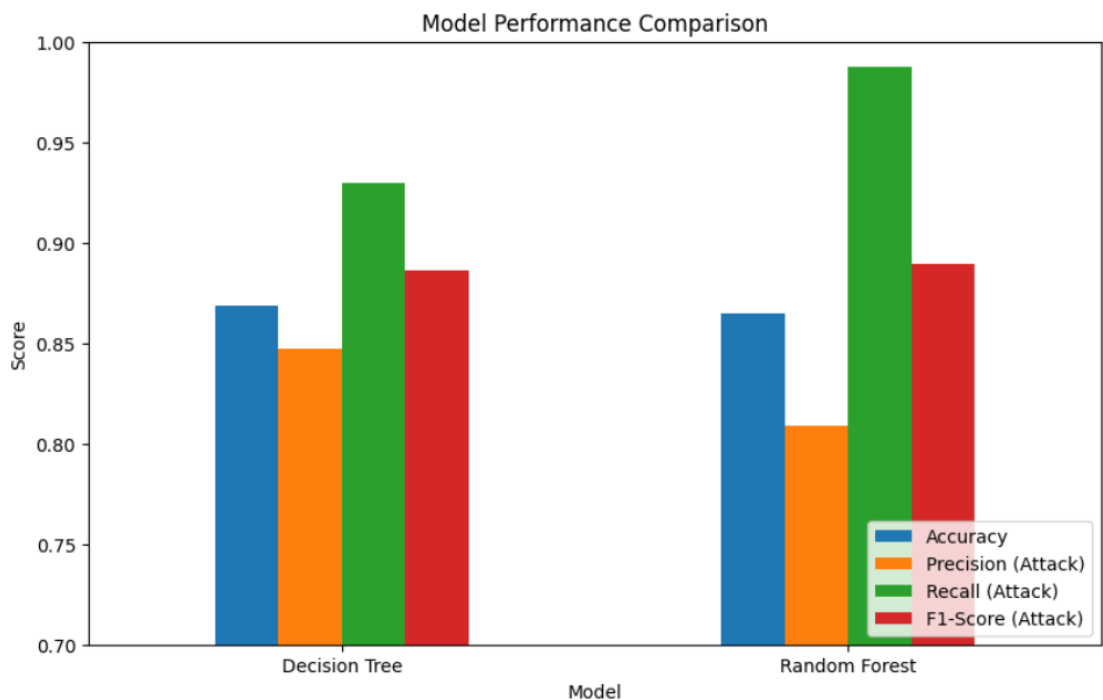
## 8. Comparative Analysis and Model Selection

The performance of both models is summarised

### Performance Comparison

| Metric             | Random Forest | Decision Tree |
|--------------------|---------------|---------------|
| Accuracy           | 86.49%        | <b>86.89%</b> |
| Precision (Attack) | 0.81          | <b>0.85</b>   |
| Recall (Attack)    | <b>0.99</b>   | 0.93          |
| F1-score (Attack)  | 0.89          | 0.89          |
| False Negatives    | <b>560</b>    | 3,199         |

Figure 4-Model Performance Comparison



False negatives are the most dangerous categorisation errors in cybersecurity because they allow harmful traffic to pass undetected. The Random Forest model misclassified only **560 attack samples**, as opposed to **3,199** for the Decision Tree. This illustrates that the Random Forest model detects attacks more reliably, despite producing more false positives.

## **Part B:**

### **1. Data Poisoning Strategy**

To evaluate the robustness of the intrusion detection model, an insider data poisoning attack was simulated by modifying the training labels before model training. The attack assumes that a malevolent employee has unauthorised access to the training pipeline and purposefully modifies the class labels contained in **y\_train** while keeping the feature values constant. As a result, the link between network traffic and its labels deviates, causing the model to learn inaccurate patterns.

Two poisoning strategies were implemented:

- **Random Label Flipping**
- **Targeted Label Flipping**

The learned Random Forest model from Part A was retrained on poisoned datasets with 5%, 10%, 15%, 20%, 25%, and 30% altered labels.

### **2. Random Label Flipping**

Random label flipping represents a general data poisoning attack where a selected percentage of training labels is randomly changed to the opposite class. Since the labels are modified randomly, both normal and attack samples may be affected.

#### **Random Label Flipping Results**

| Poison (%) | Accuracy | Precision | Recall | F1-score | False Negatives |
|------------|----------|-----------|--------|----------|-----------------|
| 5          | 0.8634   | 0.8100    | 0.9822 | 0.8878   | 806             |
| 10         | 0.8530   | 0.7995    | 0.9783 | 0.8799   | 984             |
| 15         | 0.8473   | 0.7984    | 0.9667 | 0.8746   | 1509            |
| 20         | 0.8457   | 0.8052    | 0.9494 | 0.8714   | 2292            |
| 25         | 0.8278   | 0.7915    | 0.9331 | 0.8565   | 3031            |
| 30         | 0.8127   | 0.7881    | 0.9025 | 0.8414   | 4422            |



Figure 5-Accuracy vs Poisoning Percentage

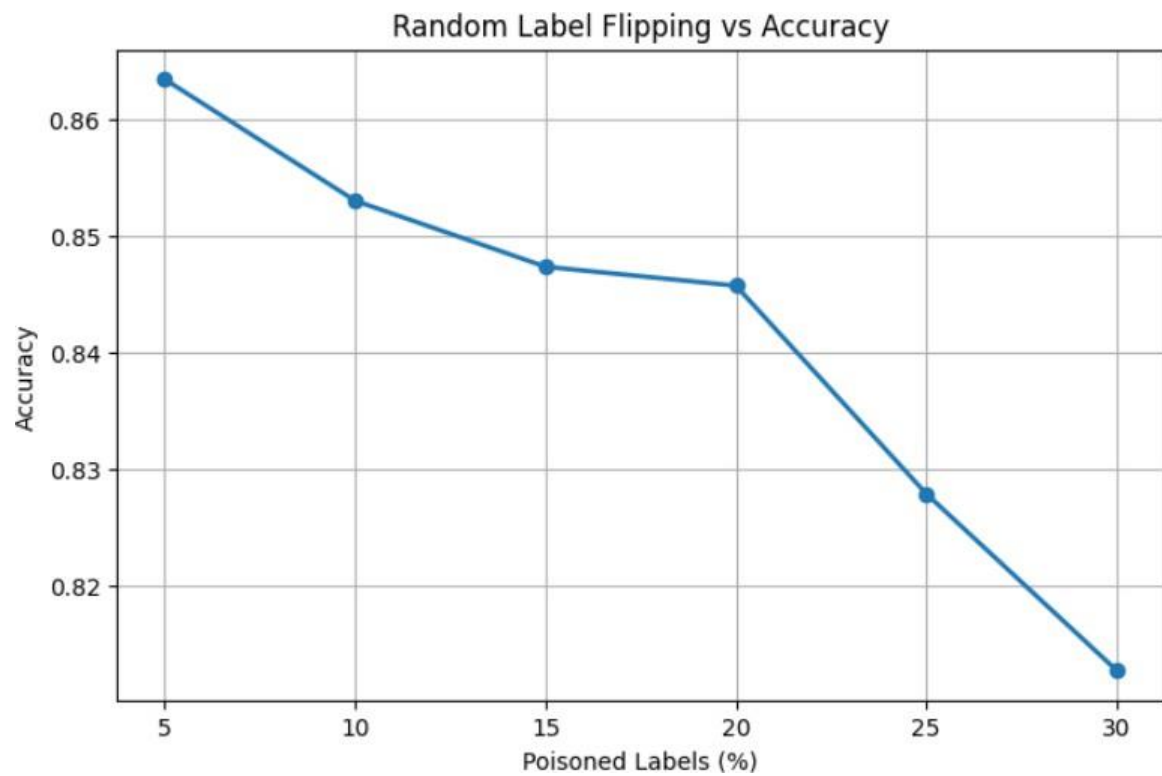


Figure c-Recall vs Poisoning Percentage

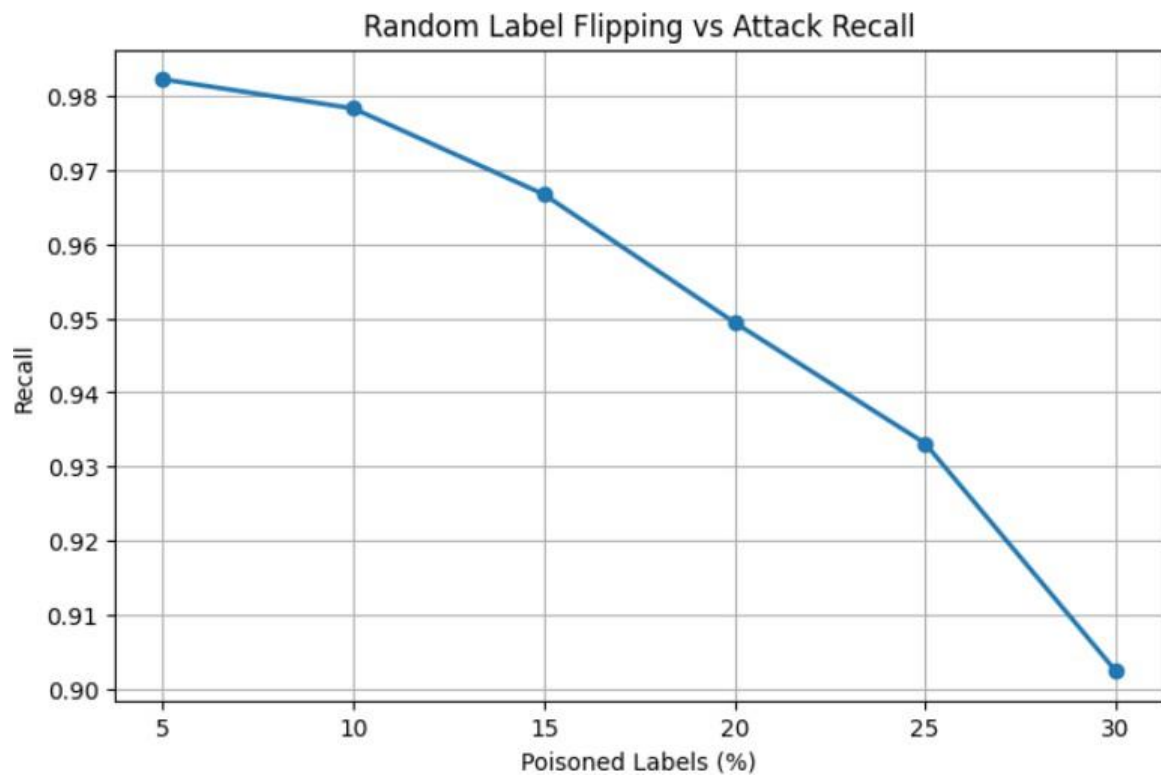
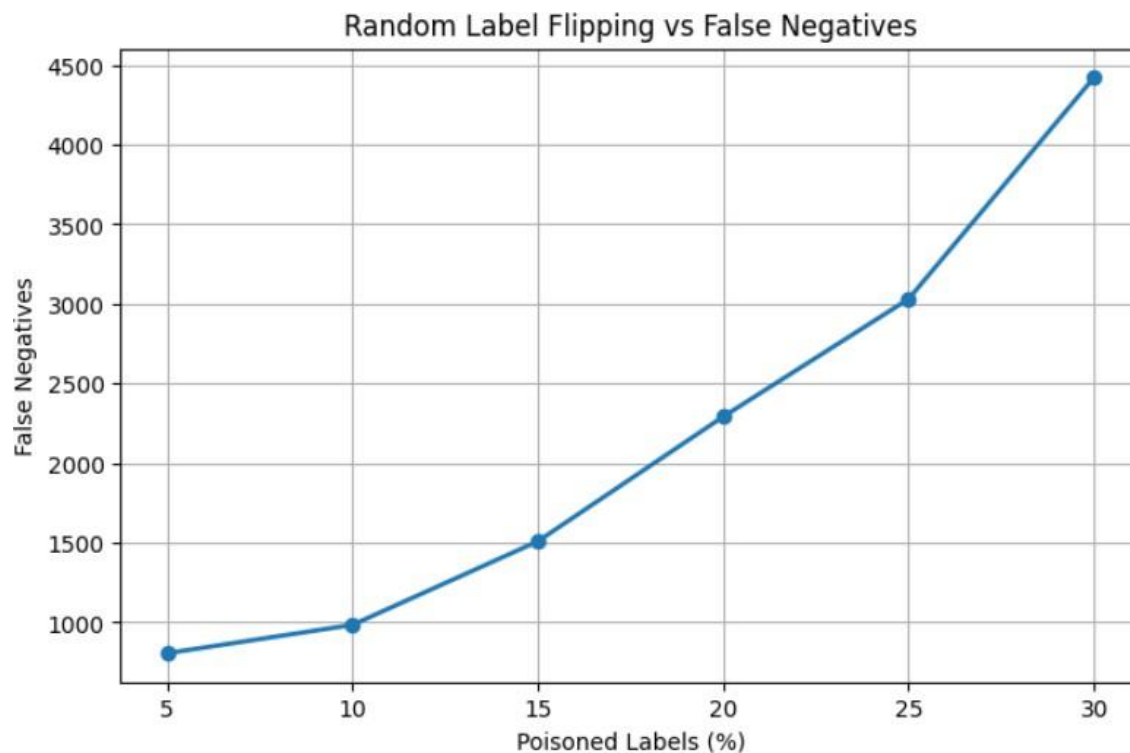


Figure 7-False Negatives vs Poisoning Percentage



The results show a gradual degradation in model performance as the poisoning percentage increased. Model accuracy dropped from **86.34%** at **5%** poisoning to **81.27%** at **30%** poisoning, and recall fell from **98.22%** to **90.25%**. The number of false negatives increased from **806** to **4,422**, indicating that more harmful network activity were misclassified as routine traffic. The results demonstrate that even moderate levels of random label manipulation reduce the model's ability to detect cyber threats.

### 3. Targeted Label Flipping

To simulate a more sophisticated insider attack, a targeted label flipping strategy was implemented. Instead of randomly modifying labels, only attack samples were selected and relabelled as normal traffic. This attack directly targets the decision boundary of the classifier and attempts to reduce its ability to recognise malicious behaviour.

Compared with random label flipping, this strategy manipulates fewer useful samples while maximising the reduction in attack detection capability.

#### Targeted Label Flipping Results

| Poison (%) | Accuracy | Precision | Recall | F1-score | False Negatives |
|------------|----------|-----------|--------|----------|-----------------|
| 5          | 0.8736   | 0.8257    | 0.9766 | 0.8949   | 1061            |
| 10         | 0.8816   | 0.8439    | 0.9631 | 0.8996   | 1673            |
| 15         | 0.8831   | 0.8541    | 0.9499 | 0.8995   | 2273            |
| 20         | 0.8812   | 0.8709    | 0.9207 | 0.8951   | 3593            |
| 25         | 0.8786   | 0.8941    | 0.8843 | 0.8892   | 5245            |
| 30         | 0.8761   | 0.9023    | 0.8690 | 0.8853   | 5940            |

### Targeted Poisoning Performance

Figure 8-Targeted Label Flipping Vs Accuracy

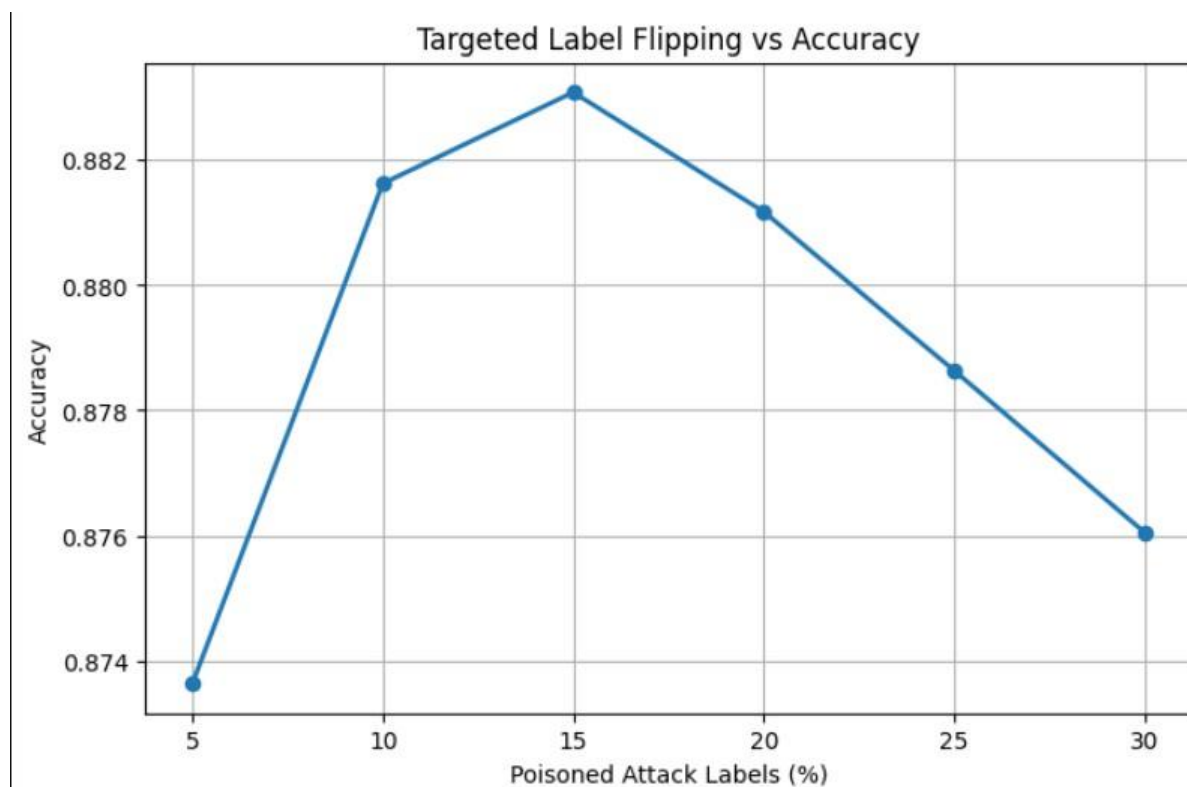


Figure 5-Targeted Label Flipping Vs Attack Recall

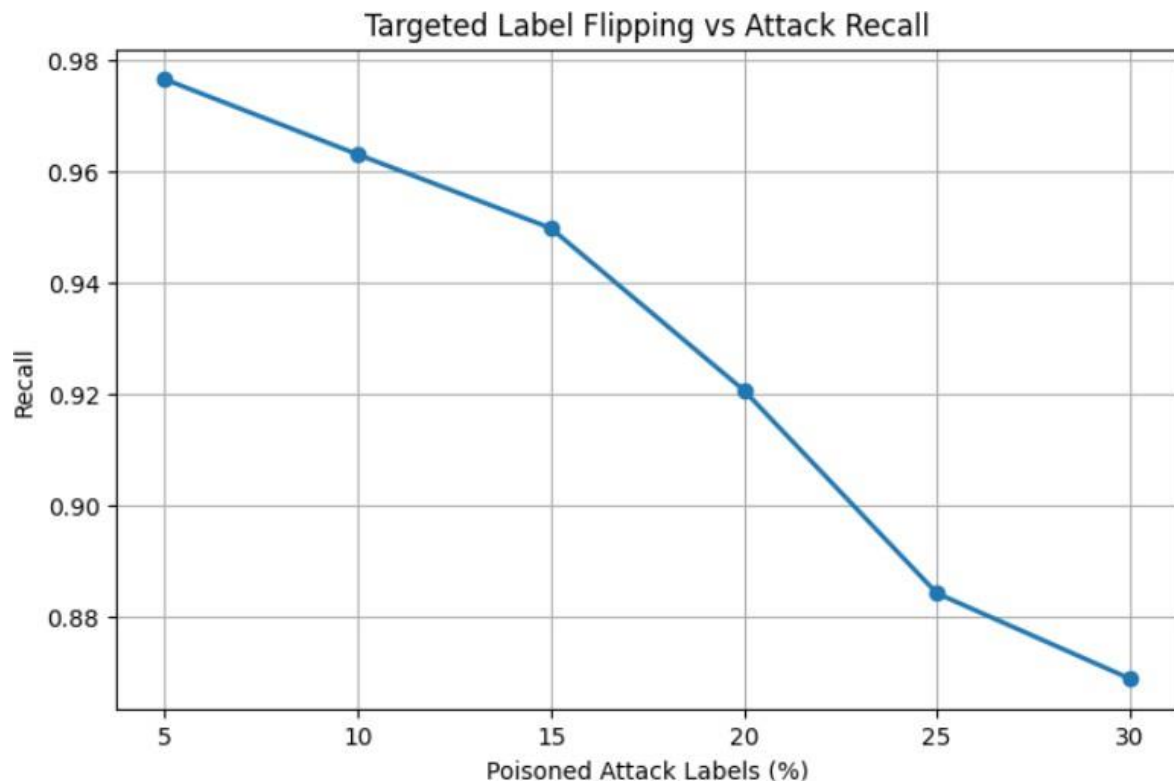
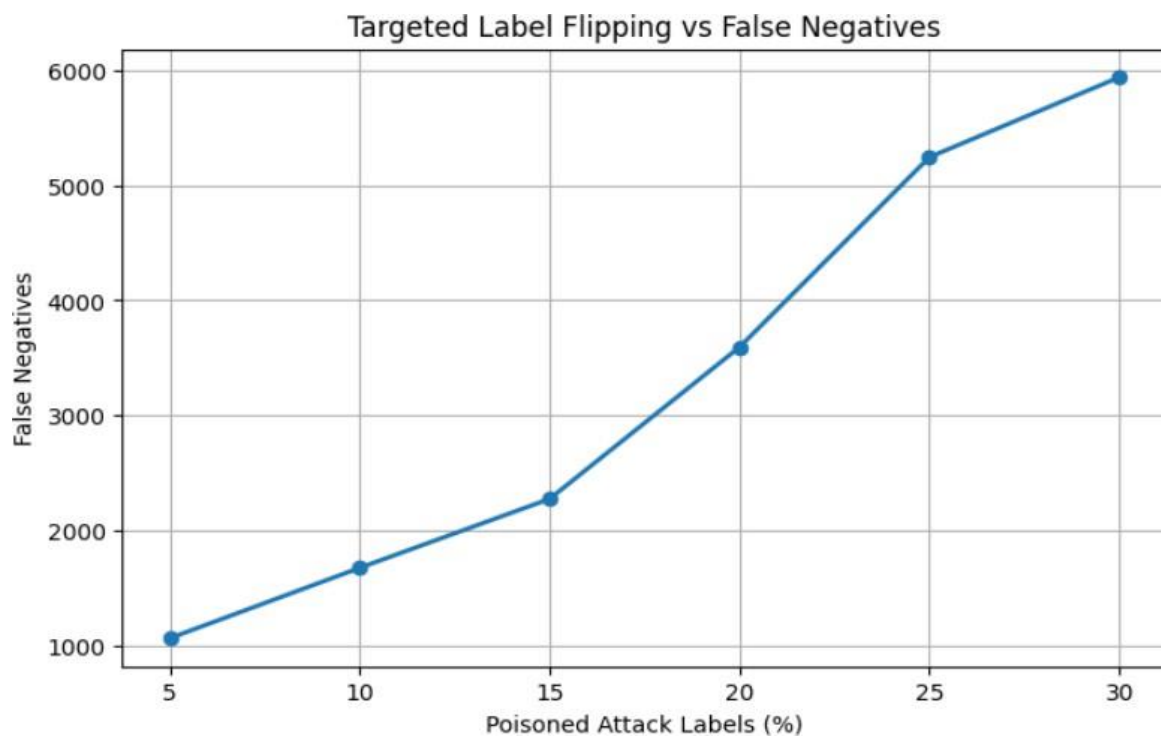


Figure 10-Targetted Label Flipping vs False Negative



The targeted poisoning attack produced a significantly different behaviour from random poisoning. Although overall accuracy remained reasonably stable throughout the experiment, the model's ability to detect malicious traffic significantly reduced. Recall decreased from **97.66% at 5% poisoning** to **86.90% at 30% poisoning**, while false negatives rose from **1,061** to **5,940**.

These findings suggest that overall accuracy alone is insufficient for assessing intrusion detection systems. A poisoned model may continue to report high accuracy while failing to detect a substantial number of cyber attacks.

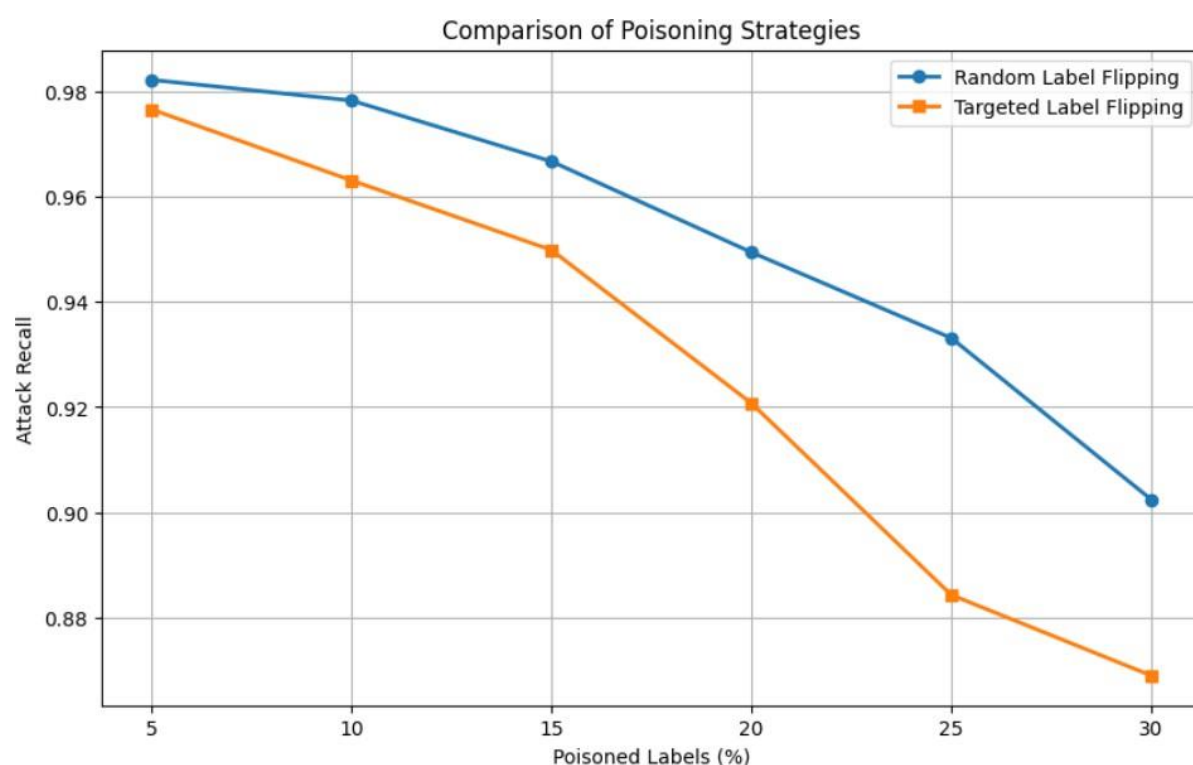
#### 4. Experimental Results

To compare both poisoning strategies, the evaluation metrics were analysed across all poisoning percentages.

**Table Comparison of Poisoning Strategies**

| Poisoning Level (%) | Random Accuracy | Targeted Accuracy | Random Recall | Targeted Recall | Random False Negatives | Targeted False Negatives |
|---------------------|-----------------|-------------------|---------------|-----------------|------------------------|--------------------------|
| 5                   | 0.8634          | 0.8736            | 0.9822        | 0.9766          | 806                    | 1,061                    |
| 10                  | 0.8530          | 0.8816            | 0.9783        | 0.9631          | 984                    | 1,673                    |
| 15                  | 0.8473          | 0.8831            | 0.9667        | 0.9499          | 1,509                  | 2,273                    |
| 20                  | 0.8457          | 0.8812            | 0.9494        | 0.9207          | 2,292                  | 3,593                    |
| 25                  | 0.8278          | 0.8786            | 0.9331        | 0.8843          | 3,031                  | 5,245                    |
| 30                  | 0.8127          | 0.8761            | 0.9025        | 0.8690          | 4,422                  | 5,940                    |

Figure 11-Comparison of Random and Targeted Label Flipping



**Figure-11** compares the attack recall achieved under random and targeted label poisoning. As the poisoning percentage increases, both strategies reduce the model's ability to detect malicious traffic. However, targeted label flipping produces a much steeper decline in recall than random label flipping. At 30% poisoning, attack recall decreases to **86.90%** for targeted poisoning compared with **90.25%** for random poisoning.

Although the targeted poisoning strategy maintains relatively high overall accuracy, it generates a substantially higher number of false negatives by misclassifying attack samples as normal traffic. This makes the attack more difficult to identify if model performance is evaluated using accuracy alone. The study found that targeted label flipping is a more efficient poisoning strategy since it minimises intrusion detection while requiring fewer strategically manipulated labels.

## 5. Critical Evaluation

The experimental results show that both poisoning procedures degrade the performance of the intrusion detection model, but their effects vary significantly.

Random label flipping inserts noise into the training dataset, gradually **decreasing accuracy, recall, and F1-score** as the poisoning percentage grows. Performance degradation is consistent, as both regular and malicious samples are affected.

Targeted label flipping is more effective because only attack labels are manipulated. This forces the model to learn incorrect representations of malicious traffic while preserving the majority

of normal samples. Consequently, the model continues to achieve high overall accuracy but becomes significantly less effective at identifying cyber attacks.

At **30% poisoning**, targeted label flipping produced **5,940 false negatives**, compared with **4,422 false negatives** under random poisoning. This confirms that strategically manipulating attack labels is a more efficient poisoning strategy and better reflects the behaviour of a malicious insider attempting to weaken an intrusion detection system.

## 6. Countermeasures

The experimental findings emphasise the necessity of safeguarding the AI training pipeline against insider attacks. Several protective steps can lower the danger of label poisoning.

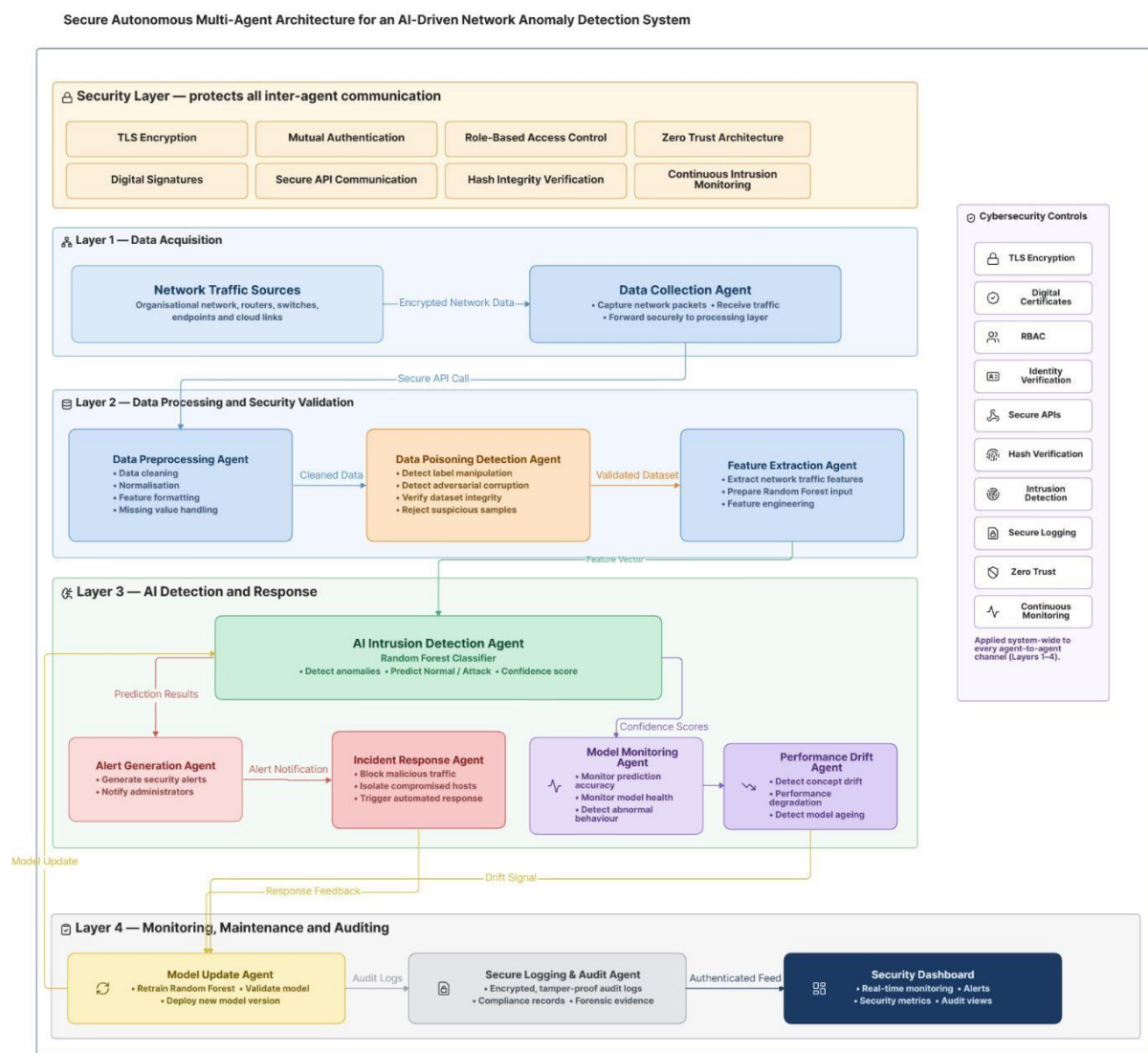
- Apply strict access control to training datasets and model development environments.
- Maintain cryptographic hashes and version control to verify dataset integrity.
- Use secure audit logs to record all modifications made to training data.
- Validate labels before model retraining using automated consistency checks.
- Monitor unusual changes in model performance, particularly recall and false negative rates.
- Retrain models only after verifying the integrity of the training dataset.

# Part C:

## 1. Introduction

A secure autonomous multi-agent architecture was created to facilitate the deployment and maintenance of the AI-based network anomaly detection system built in Parts A and B. Independent agents handle data, identify intrusions, monitor models, and communicate securely using authentication, encryption, and access control protocols.

Figure 12-Secure Autonomous Multi-Agent Architecture for AI-Driven Network Anomaly Detection System



## 2. Professional Architecture Diagram

**Figure-12** illustrates the proposed architecture, organised into four operational layers: **Data Acquisition**, **Data Processing and Security Validation**, **AI Detection and Response**, and



**Monitoring, Maintenance and Auditing.** A dedicated security layer protects all communication channels between agents using standard cybersecurity controls.

### 3. Agent Description

| Agent                          | Primary Responsibility                                      |
|--------------------------------|-------------------------------------------------------------|
| Data Collection Agent          | Collects network traffic and securely forwards data.        |
| Data Preprocessing Agent       | Cleans and normalises network traffic.                      |
| Data Poisoning Detection Agent | Detects manipulated labels and validates dataset integrity. |
| Feature Extraction Agent       | Generates feature vectors for the Random Forest model.      |
| AI Intrusion Detection Agent   | Classifies traffic as normal or malicious.                  |
| Alert Generation Agent         | Generates security alerts.                                  |
| Incident Response Agent        | Performs automated mitigation actions.                      |
| Model Monitoring Agent         | Monitors prediction accuracy and model health.              |
| Performance Drift Agent        | Detects concept drift and performance degradation.          |
| Model Update Agent             | Retrains and deploys updated models.                        |
| Secure Logging & Audit Agent   | Maintains encrypted audit logs.                             |
| Security Dashboard             | Displays security metrics and system status.                |

### 4. Coordination Between Agents

The **Data Collection Agent** sends captured traffic to the **Data Preprocessing Agent**, where it is cleansed and validated by the Data Poisoning Detection Agent. The **AI Intrusion Detection Agent** transforms validated data into feature vectors and classifies them.

Prediction findings activate the **Alert Generation and Incident Response Agents**, while the **Model Monitoring** and **Performance Drift Agents** continuously monitor model behaviour. When the **Model Update Agent** detects degradation, it retrains the model. The **Secure Logging & Audit Agent** records all system actions and displays them on the Security Dashboard.

## 5. Security Mechanisms

| Security Mechanism               | Purpose                                |
|----------------------------------|----------------------------------------|
| TLS Encryption                   | Protects inter-agent communication.    |
| Mutual Authentication            | Verifies agent identities.             |
| Role-Based Access Control (RBAC) | Restricts agent permissions.           |
| Digital Signatures               | Ensures message authenticity.          |
| Hash Integrity Verification      | Detects unauthorised modifications.    |
| Secure API Communication         | Secures agent interfaces.              |
| Secure Logging                   | Maintains tamper-resistant audit logs. |
| Zero Trust Architecture          | Continuously verifies all agents.      |
| Continuous Intrusion Monitoring  | Detects attacks against the system.    |

## 6. Threat Mitigation

The architecture uses numerous protection methods to limit the danger of insider attacks and AI security risks. The **Data Poisoning Detection Agent** validates training data prior to model changes, while **TLS, digital signatures, RBAC, and Zero Trust** ensure safe inter-agent communication. When inappropriate behaviour is detected, continuous monitoring and performance drift detection allow for automatic retraining.

## 7. Relationship with Parts A and B

The proposed architecture uses the **Random Forest intrusion detection model** built in Part A to combat the label poisoning threats analysed in Part B. The **Data Poisoning Detection Agent, Model Monitoring Agent, and Performance Drift Agent** all contribute to detecting manipulated training data, monitoring prediction quality, and ensuring dependable intrusion detection performance.

## 8. Conclusion

The proposed multi-agent architecture integrates **AI-powered intrusion detection** with secure communication, continuous monitoring, automatic reaction, and model maintenance. These components enhance the security, dependability, and trustworthiness of the deployed network anomaly detection system while mitigating the effects of data poisoning and other cyber attacks.